

DEVOPS · PLATFORM · 2026-09-10

forklift Go에서 Rust로

3주간 다섯 번 리소스를 올리며 버티다 런타임을 바꿨다. 상주 메모리 86% 감소.

전환 2026-09-09 01:32 KST

버전 0.12.2 → 0.13.0

런타임 Go → Rust 1.98.1

01 · 포팅 규모

이틀 만에 62.8k 라인

Go 275 파일 62.8k LOC 를 Rust 276 파일 92.3k LOC 로 옮겼다. 재작성이 아니라 모듈 대 모듈 1:1 포팅 · 커밋 1개 · 테스트 714건.

숫자 넷

—61%

리더 파드 rss p50 · 138 → 55 MiB

—75%

CPU request · 1000m → 250m

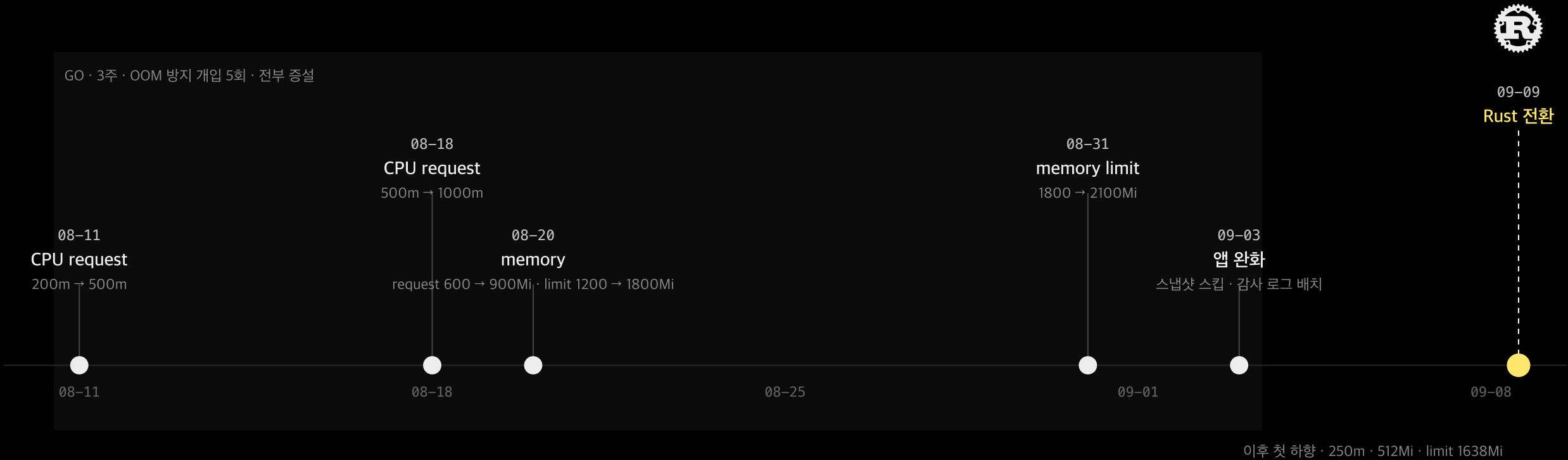
—22%

메모리 limit · 2100Mi → 1638Mi · request 900Mi → 512Mi

—70%

리더 파드 working set p50 · 640 → 194 MiB

OOM 을 막으려 3주 동안 다섯 번 올렸다



DevOps Engineer 가 매일 메모리 지표를 보며 limit 에 닿기 전에 올렸다. 요청이 늘면 다시 올려야 하는 구조.

CI/CD 전체가 지나가는 경로

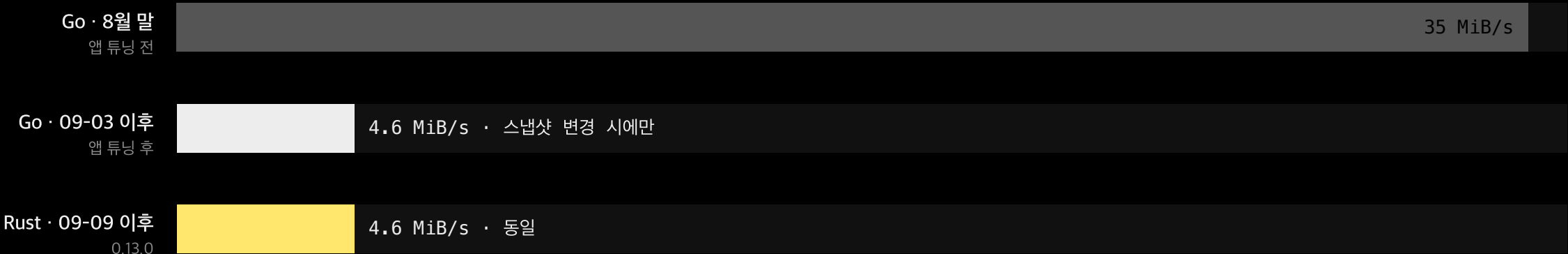


working set 은 limit 에 붙고, rss 는 부하마다 튀었다



OOMKilled 0건. 사고는 없었지만 GC 가 힘을 돌려주는 시점을 앱이 정하지 못했다.

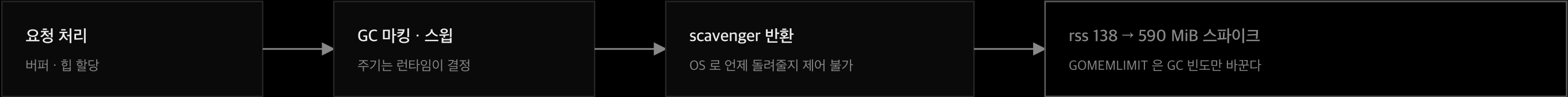
I/O 는 앱 튜닝으로 잡혔다. 메모리는 남았다



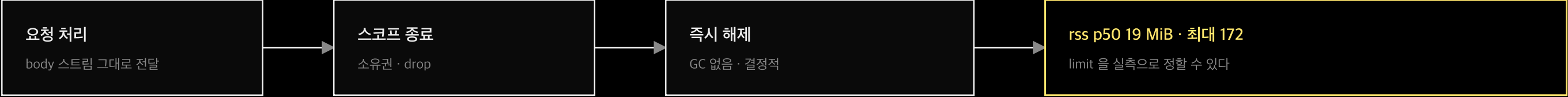
스냅샷은 data_version 변경 시에만, 다운로드는 ETag 변경 시에만, 감사 로그는 256건 배치, rss 스파이크는 그대로.

GC 가 정하던 반환 시점을 코드가 정하게

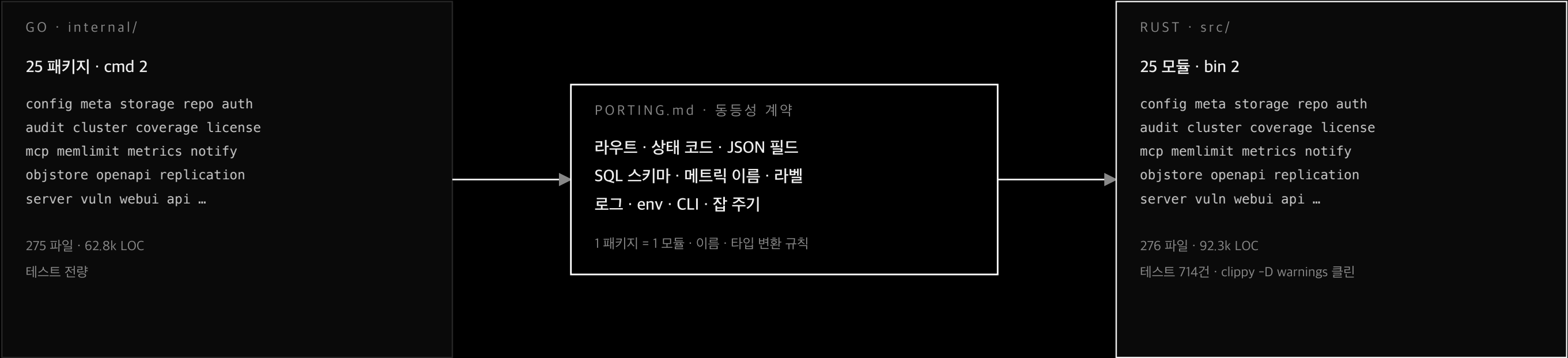
GO · 0.12.2



RUST · 0.13.0



재작성이 아니라 1:1 포팅



25

모듈 1:1

715

파일 · +98.8k / -63.9k

714

테스트

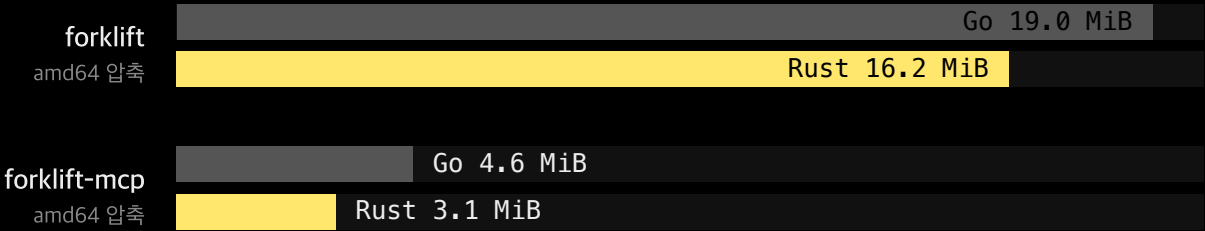
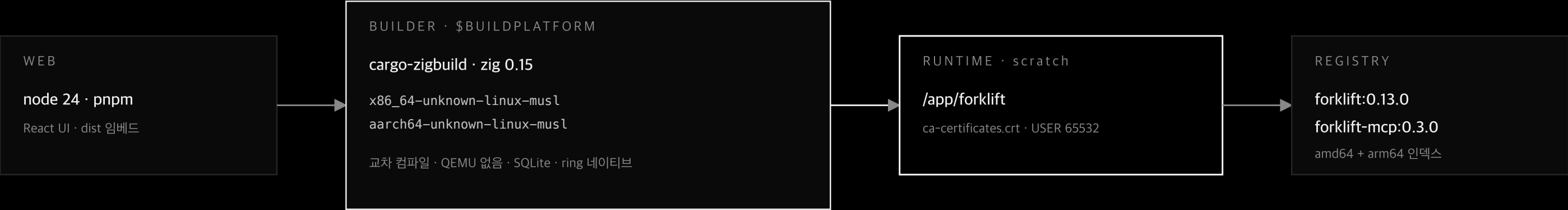
1.98.1

Rust · edition 2024

표준 라이브러리 자리에 들어간 것

HTTP Go net/http · goroutine tokio 1.53 · hyper 1 · axum 0.8 HTTP/1 · HTTP/2 · graceful shutdown		
CLIENT Go net/http reqwest 0.13 · rustls (ring)	AUTH go-oidc · bcrypt openidconnect 4 · bcrypt · hmac	
METADATA modernc sqlite rusqlite 0.40 · bundled	BLOB aws-sdk-go-v2 aws-sdk-s3 1.145 · sigv4	
KUBERNETES client-go kube 4.2 · Lease	MCP mcp-go rmcp 3.2	OBSERVABILITY client_golang · pprof prometheus 0.14 · tracing
BINARY Rust 1.98.1 · musl 정적 링크 · scratch · CA 번들만 동봉 · 셸 · libc 없음 왼쪽 회색 = Go · 흰색 = Rust		

QEMU 없이 두 아키텍처, scratch 위에 바이너리 하나



데이터 마이그레이션 없이 이미지 태그로

1차 완화

Go · I/O 완화



09-03

Rust 커밋 · 0.13.0

chart 0.12.0 · 레지스트리 미러



09-08

프로덕션 전환

rollout · Lease 이전 · 무중단



09-09 01:32

리소스 right-sizing

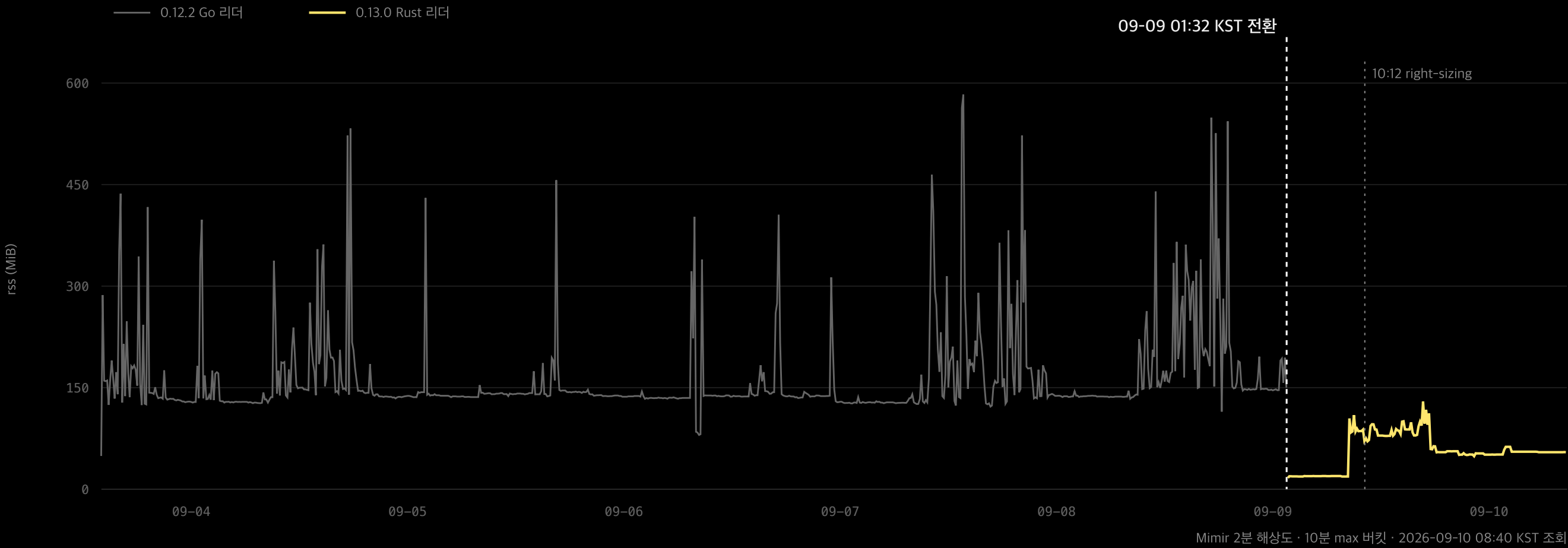
250m · 512Mi · limit 1638Mi



09-09

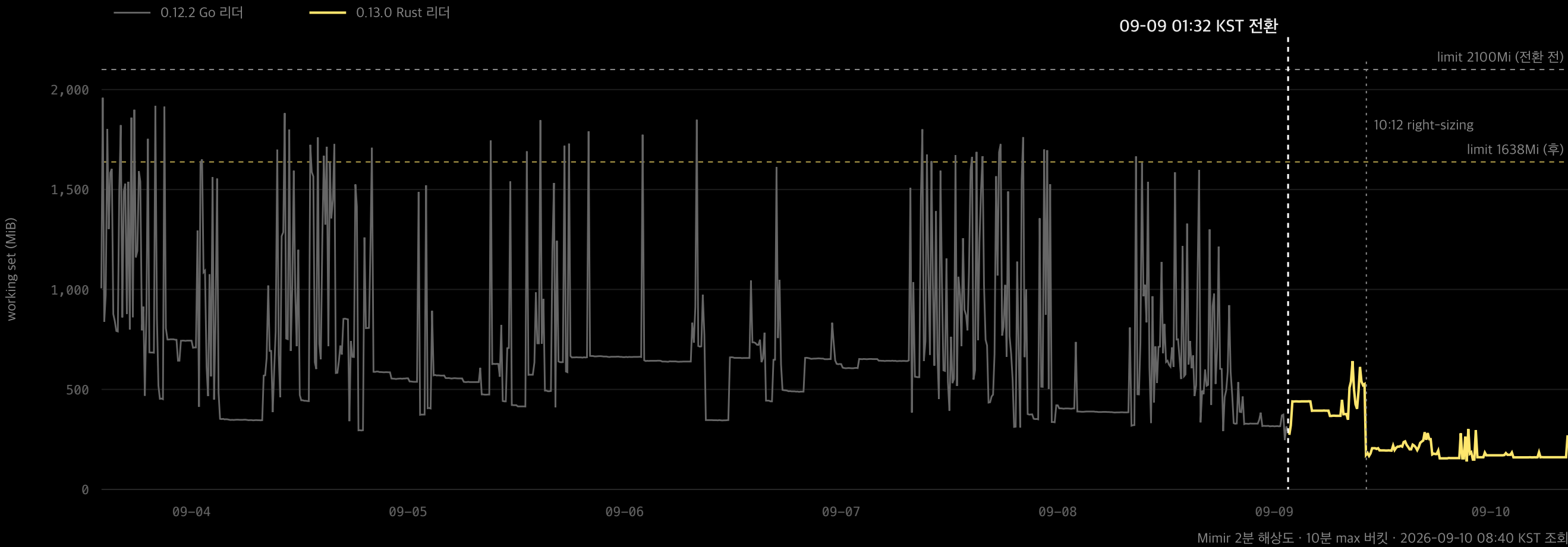
호환 유지: HTTP 라우트 · SQLite 마이그레이션 · 세션 쿠키 · 액세스 토큰 · 저장소 포맷. 롤백은 태그 되돌리기.

138 → 55 MiB. 업무 시간대 피크도 583 → 130



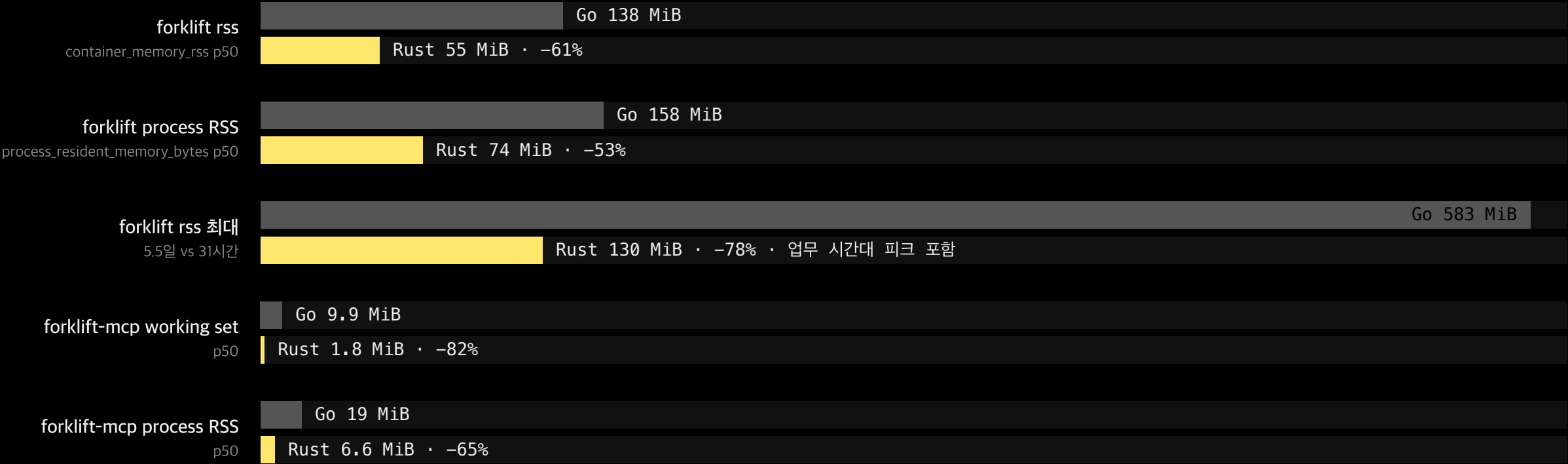
Go 5.5일 p50 138 · p95 313 · 최대 583 MiB → Rust 31시간 p50 55 · p95 97 · 최대 130 MiB. 같은 새벽 유티 구간만 보면 137 → 55 MiB, 같은 업무 시간대(09-17시) p50 은 197 → 86 MiB.

working set 은 page cache 의 그림자

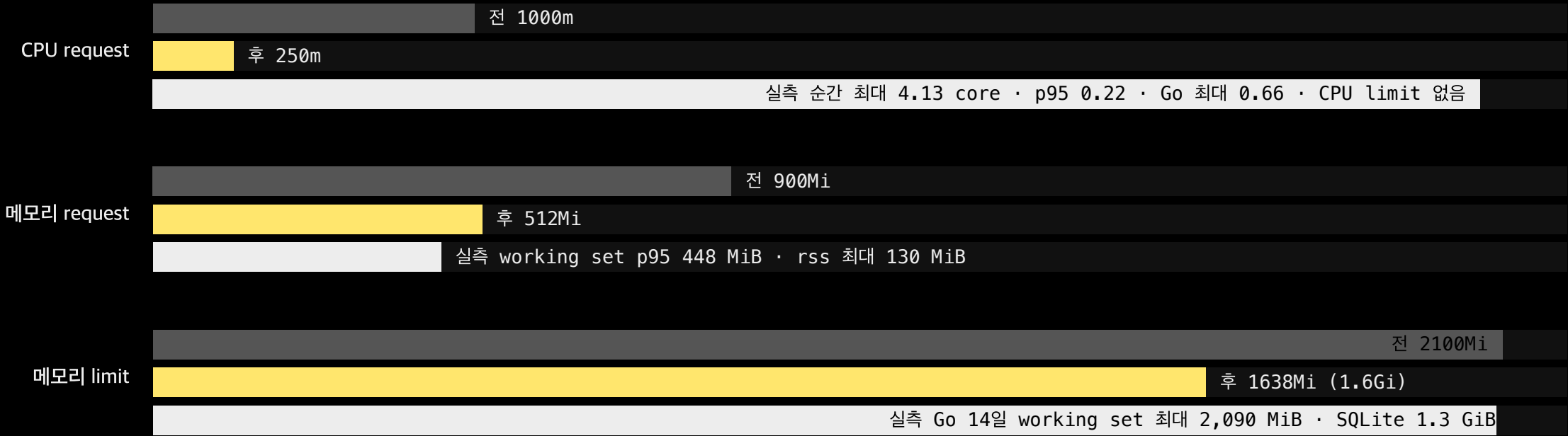


working set = rss + active page cache. Go p50 640 · p95 1,638 · 최대 1,959 MiB → Rust p50 194 · p95 448 · 최대 642 MiB. 같은 새벽 유희 구간 389 → 161 MiB. limit 에 불던 구간이 사라졌다.

앞뒤 비교

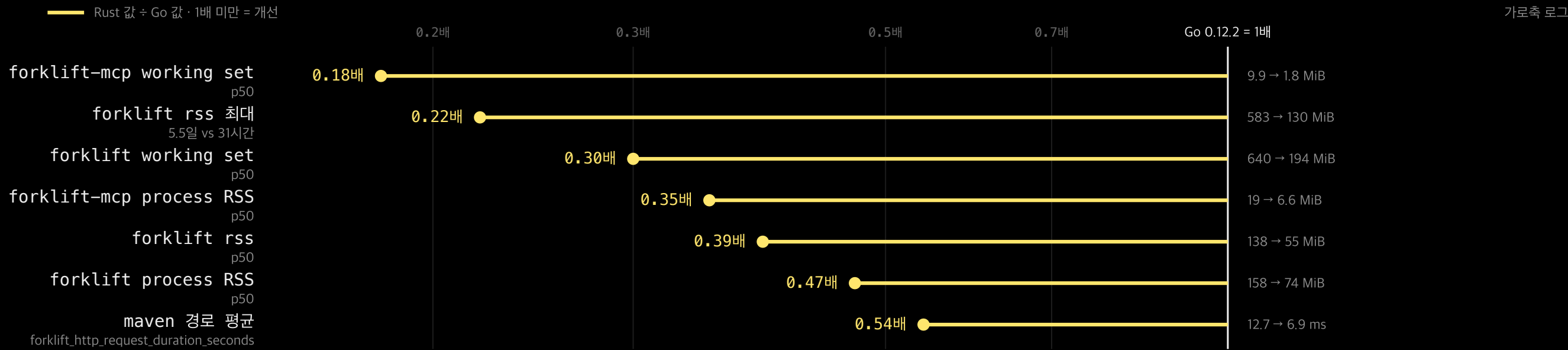


실측 위에 request 와 limit 을 다시 놓았다



회색 전 · 노랑 후 · 흰색 실측. CPU 행은 0~4400m 축, 메모리 행은 0~2200Mi 축. limit 은 page cache 상한만 남는다. CPU 는 limit 이 없어 순간 4.13 core 까지 올라간다. 1Gi 까지 내리면 SQLite hot page 가 반복 회수되어 I/O 가 는다.

동일 워크로드에서 리더 파드 메모리를 60~78% 줄였다



비교 구간 Go 0.12.2 5.5일(09-03 14:02 ~ 09-09 01:32) vs Rust 0.13.0 31시간(09-09 01:42 ~ 09-10 08:40). 리더 파드 · Mimir 2분 해상도 · 10분 max 버킷. 요청량은 동일 수준(7,442 → 6,639 req/h), 업스트림 응답은 오히려 개선(37.8 → 29.6 ms).

이 데크이 참조한 자료

- github.com/younsl/forklift
- CHANGELOG 0.13.0 · PORTING.md · Dockerfile · docs/metrics.md
- Helm chart · oci://ghcr.io/younsl/charts/forklift

- Prometheus 장기 저장소(Mimir) · cadvisor · process_* · forklift_* · 2026-09-10 08:40 KST 조회
- goldilocks VPA · crane manifest
- 차트는 2분 해상도 데이터를 10분 max 버킷으로 그린 인라인 SVG